

ADR-008 — Serving while indexing, and the one answer that must never be `200`

- **Status:** Accepted and **implemented** — the server half, as scoped below. Approved by Jay on 2026-11-02 after the measurements in this document. Not deployed at the time of writing.
- **Date:** 2026-11-01
- **Deciders:** Jay
- **Supersedes:** none. Fixes a defect in the start-up path chosen by ADR-002 ("the catalog is an in-memory index, rebuilt at start"), and hardens increment 1 of ADR-004.

Context

The `0c6fa84` deploy took the live server **hard down for 10m47s**. Not slow — `/healthz` was *connection-refused* for the whole window, because there was nothing listening to refuse it politely.

What the start-up path actually does

`cmd/yarg-song-server/main.go`, `run()`:

1. line 311 — `ix, err := library.Build(opt.Songs)`, synchronous, walks the entire library
2. ~150 lines of cheap setup (pack cache, features, logging)
3. line 465 — `srv.ListenAndServe()`

The socket is not bound until the index is complete. From outside, **"indexing a large library" and "crashed on boot" are the same observation**, which is also why the deploy script's health patience has to be 1800s: a long index cannot be distinguished from a hang by anything a deployer can see.

`library.Build` is one single-threaded `filepath.WalkDir` (`internal/scan/library.go:38`), and per song `internal/scan/scan.go:138` computes `packageHash` — SHA-256 over the sorted `<lowercased name>:<sha256>` lines, which means **it reads every byte of every file in every song, audio included**, on every restart. The cost is proportional to the bytes in the library and is paid in full each time the process starts.

The measurements

Songs	Index time	Outage	Per song
23 (v0.1.3, 2026-09-15)	~30 s	~30 s	~1.3 s
1,369	3m39s	3m39s	0.160 s
3,067 (`0c6fa84` , tonight)	10m40.186s	**10m47s**, 21:21:31Z–21:32:18Z	0.209 s
4,085 (probe, same night)	16m34.49s	n/a — ran beside the live server	0.244 s
4,085 (**live**, `0c6fa84`)	16m48.052s	**16m53s**, 00:20:21Z–00:37:14Z	0.247 s

The probe predicted the live cost to within 14 seconds — 16m34.49s against 16m48.052s on the real container, 1.4% apart. That validates the method as much as the number: *a future restart's cost can be measured without taking an outage*, by indexing the same library in a throwaway instance beside the running one. It means the next decision about this does not have to be paid for with downtime, which is worth more than either figure.

The fourth row is a **throwaway instance on port 8123** indexing the enlarged library while 8099 kept serving, so it carries no outage figure. It also ran a *different binary* — a cross-compiled `v0.1.2-40-gfab8c90` , not the live `0c6fa84` . The figure is taken as transferable because the only Go change in that range is `internal/scan/extract.go` 's `.rar` directory handling and the indexed set contains no `.rar` at all, but it is a different build and this document says so rather than quietly comparing across it.

"Roughly linear in song count" is a floor, not a model. Across the three real rows per-song cost rose **52%** — 0.160 → 0.211 → 0.244 s — so 3× the songs cost **4.55×** the time. Linear extrapolation *understates* every projection made from it. The composition explains it, and the composition is measured (below): the 1,698 songs added tonight are loose folders, and a loose folder costs more to index than a single-file container because every file in it must be opened, read and hashed.

What the indexed set is made of — measured, 2026-11-01

The coordinator's read was that the cost is walking and hashing **loose folders**, not decompressing archives, offering `archives: []` from `/api/v1/library` as the evidence.

The conclusion is right and that evidence does not carry it. `Archives()` (`internal/library/library.go:242`) is computed from `worthUnpacking` , which is `SourceZip || Source7z || SourceRar` **only** — `.sng` and `.yargsong` are excluded *by design*, and the function's own comment records 481 `.yargsong` files in this very library. So `archives: []` is silent about 15.7% of the catalogue.

Counted properly, from `source_path` across all 3,067 entries:

Shape	Count	Share
loose folder	2,586	84.3%
`.yargsong`	481	15.7%
`.zip` / `.7z` / `.rar`	**0**	0%

(The first run of that count was wrong and is worth recording: it classified with `os.path.splitext`, and song folders are full of dots — "Mr. Big - To Be With You", "ii (2x bass pedal expert+)" — so ~190 folders were booked as bogus "extensions" and the loose-folder count came out 2,397. Classify against the known container extensions, not against the tail of a name.)

So: **nothing in the indexed set has to be decompressed** — confirmed, but because `.sng` and `.yargsong` are *stored* (there is no `flate`, `gzip` or `zlib` anywhere in `internal/sng`), not because `archives:` was empty.

That distinction is the one the coordinator predicted would matter, and it does: those 481 entries are **single files with one size and one mtime**, which is a far cheaper and far safer cache key than a loose folder whose hundreds of files can each change independently. A differential index would get 15.7% of this library almost free and would have to work for the other 84.3%.

This is more urgent than when it was raised

`/api/v1/library` still reports `built_at: 2026-11-01T21:21:31.894Z` — the start-up index. Eight further packs and the Rock Band 1 DLC set have been unpacked onto the disk since, and the live index does not contain them. **The next restart therefore indexes more than 3,067 songs**, and at 0.209 s/song and rising it will beat 10m47s.

The crux: an empty library and an unbuilt one look identical

The obvious fix — bind the listener first, swap the index in when it is ready — has the machinery waiting for it. `library.Store.Replace` is the atomic swap, and `api.OnUnpacked` already uses it correctly: it builds a *new* index while the old one keeps serving, then swaps. **The rebuild path is already non-disruptive. Start-up is the one path that blocks.**

But "bind early and serve the index you have" is not a smaller version of the fix. It is a worse bug, and it destroys data on someone else's machine.

`internal/syncclient/syncclient.go:443`, `prune()`:

```
all, _, err := postHave(ctx, opt, base, nil)   // the server's FULL set
...
for h := range local {
    if _, ok := onServer[h]; ok { continue }
    os.Remove(filepath.Join(opt.Dest, h+".sng"))
}
```

A server that answers `200` with an empty list while it is still indexing is telling the truth about its index and a lie about its library. A client running with `--prune` believes it and **deletes the player's entire songs folder**. A ten-minute outage is an inconvenience; this is silent, remote, irreversible data loss, and it would be *caused by the fix*.

The repo already states this principle, in `run()`'s refusal to start on a missing library: *"The server is up and has no songs" and "the path is wrong" look identical from a client.* An unbuilt index served as an empty one is that same ambiguity, handed to the one caller that acts on it destructively.

What rescues the server-only fix: the shipped client already fails closed

`postHave` (`syncclient.go:260`) rejects any non-200:

```
if resp.StatusCode != http.StatusOK {
    return nil, 0, fmt.Errorf("syncclient: POST /api/v1/have returned %s", resp.Status)
}
```

and `Run` calls it **first**, at line 172, before any download and long before `prune`:

```
missing, total, err := postHave(ctx, opt, base, have)
if err != nil {
    return nil, err
}
```

So a **503 aborts the sync immediately, downloads nothing, and never reaches `prune`.**

That is the finding that settles the shape of this change, and it corrects the brief this ADR was written from. The server half and the client half do **not** have to land together — *provided the server answers with a status, not with an empty body*. The already-released client is safe against a 503 today, by construction, because it fails closed. It is unsafe against exactly one thing: a `200` that under-reports the library.

The invariant, stated once: `POST /api/v1/have` must never answer `200` while the index is incomplete. Everything else in this ADR follows from that sentence.

Note what this means for the genuinely-empty case: a server with a real, fully-indexed, empty library *should* answer `200 []`, and a pruning client *should* then delete everything, because that is correct. The distinction between "nothing yet" and "nothing at all" is carried by the **status code**, which is why it cannot be carried by the payload.

Decision

Bind the listener before building the index, and gate every index-dependent answer on readiness — with a status code, never with a partial payload.

1. **Build in the background.** Bind and serve immediately; run `library.Build` in a goroutine and publish with the existing `Store.Replace`. No new concurrency primitive: this is the path `OnUnpacked` already takes.

2. **Index-dependent endpoints return `503` with `Retry-After` until the first index is published.** As built, that is `POST /api/v1/have`, `GET /api/v1/songs`, `GET /api/v1/songs/{hash}`, `GET /song/{file}`, `GET /api/v1/library`, `POST /api/v1/check` (part of its answer is "do we already have this song") and `POST /api/v1/unpack` (it needs the library root *and* writes into a tree being walked). `/version` and `/api/v1/features` stay available — they do not depend on the index, and during a seventeen-minute start-up they are exactly what an operator wants to be able to ask. **Correction to this ADR's first draft, found by reading the code rather than reasoning about it:** it listed *the browse page* here. The browse page is static HTML that fetches its data from the API (`internal/httpapi/web.go`), so it carries no claim about the library and is served as normal; its own fetches get the 503. Gating it would have hidden

the diagnostic page during exactly the window it is useful. The refusal is reached through one helper, `Server.index`, so a handler cannot answer a question about the library without passing the readiness check, and the route list above is pinned by a test that fails if a new route is added and left ungated.

3. **Publish once, not progressively.** The index becomes visible when it is complete. A progressively-published index makes `/have` answer an incomplete truth, which is the prune hazard again arriving by a different road, and it makes `package_hash` ETags appear and change under clients mid-sync. The gate is binary on purpose.

4. `/healthz` **must distinguish "up, indexing" from "ready", and the deploy gate must assert ready.** Today it is `internal/httpapi/api.go:127` and writes `ok` unconditionally. This is the part that must not be split from item 1: bind the listener earlier without fixing `healthz`, and the deploy script's health loop starts passing the moment the socket opens — on a server that cannot serve a single song. **That converts tonight's visible eleven-minute outage into an invisible one, which is worse than leaving it alone.**

5. **The client change is legibility, not safety, and is therefore separable.** `yarg-sync` against an indexing server today fails with `POST /api/v1/have returned 503 Service Unavailable` — correct, and unreadable. It should say the server is still indexing and, for a long-running sync, wait and retry rather than exit. Worth doing; not a prerequisite for items 1–4.

Which increment this lands in

Increment 1 of ADR-004 — the managed mirror folder — which has already shipped. This is a correctness fix to the increment that is live, not a step toward 2 or 3. It touches no `YARG.Core` seam, needs nothing from upstream, and is not blocked on anybody's answer. Increment 2 is recommended for skipping and increment 3 is a performance feature; neither is relevant here. The `prune` hazard exists *because* increment 1 put a real sync client in front of this server, and that client is the only caller that can turn a server-side ambiguity into deleted files.

How it was built, and how the tests were proved able to fail

`library.NewPendingStore` is a store with no index yet, and `Get` returns **nil** in that state rather than an empty index. That is the load-bearing choice: an empty index is a *lie a caller acts on*, and a handler that forgets the readiness check panics into a 500 for one request instead of quietly telling a pruning client that the library is empty. Failing loudly beats answering "empty" quietly.

`internal/httpapi/indexing_test.go` pins the behaviour, and each of its tests was **reintroduced against its own defect** before its green was trusted — per *tune a regression test against the defect, or it is decoration*:

Defect reintroduced	Test that went red
`index()` returns an empty index instead of refusing	refuses-while-indexing, **and** have-never-200
`/healthz` always answers `ok`	healthz-says-indexing-then-ok
`index()` refuses even when ready	empty-library-still-answers-200

All four pass, all four were observed failing, and the tree was restored and re-verified green. The last row is the one that stops the distinction being collapsed: a server that really has no songs must answer `200` with an empty list, and a pruning client is *right* to act on it.

`go build`, `go vet`, `gofmt -l` and `go test ./... -count=1 -race` are all clean.

Consequences

- **Hard downtime goes to zero**, regardless of library size, and stays zero as the library grows. That is the whole point: it decouples availability from index cost instead of making index cost smaller.
- **Clients and monitors get a readable answer** during start-up. A 503 with `Retry-After` is something a client, a reverse proxy and a human can each act on correctly; connection-refused and an empty `200` are each unreadable, in opposite directions.
- **The window gets *longer*, not shorter.** Today the server is unreachable for eleven minutes; afterwards it is reachable and refusing song requests for eleven minutes. This is a better failure, not a faster one, and it should not be described as a performance fix.
- **A deploy gate that asserts only "the socket is open" becomes wrong.** Item 4 is load-bearing and `docs/DEPLOY-VAULT2.md` needs updating in the same change.
- **The last hard restart is the one that ships this, and it is the argument for not waiting.** The fix cannot help its own deployment: the binary being replaced is the one without it, so that restart is hard-down for the full index like every restart before it. Every restart *after* it is soft. So the cost of this change is one more outage of the current size, and the cost of deferring it is that same outage at whatever size the library has reached by then — which is rising, and rising faster than song count because the mix is shifting toward loose folders. There is no restart at which this gets cheaper to adopt.
- **Indexing cost is untouched and still scales with library bytes.** Three candidates, all deliberately out of scope here, and they are listed cheapest-and-safest first:
- ***Stop hashing every file twice.*** `scan()` calls `collectAssets` and then `packageHash` over the same `names`, and both go through `assetFor`, which is `io.Copy(sha256.New(), f)` — a full-byte read. `collectAssets` hashes the album art, the stems and every leftover file; `packageHash` then hashes **every** file again, the stems included. Since audio is almost all of the bytes, **an index reads the library about twice**. That matches the only profile anyone has taken: `/proc/<pid>/io` on the 4,085-song probe reported `rchar` **234,002,202,475** — 234 GB of `read()` bytes — with `read_bytes` 0 because shfs/FUSE hides the block layer, and 753s of CPU against 974s elapsed. *That 77% was first written down here as "compute-bound rather than IO-blocked", and that overstates it.* CPU/wall = **0.773 is mean parallelism below one**: the index is effectively **single-threaded with about 23% of its wall time spent waiting**. Which matters, because it names a *second* lever — hashing files concurrently — that is independent of hashing them once. Whether the walk can be parallelised safely is unmeasured and not proposed here; the point is only that **~2× is not the ceiling**. **The doubling is quantified rather than inferred by halving that number.** The catalogue carries every asset's size, so the library total can be summed exactly, with no disk I/O at all: **118,083,305,071 bytes — 118.08 GB across 27,541 asset files**, 28.9 MB and 6.7 files per song over 4,085 songs. Two reads of that is a floor of 236.17 GB. The raw `rchar` above gives 1.982, which is *below* that floor and so contradicted the model — the chart and `song.ini` are not in `assets`, and `packageHash` reads them on top of the two asset passes, so the ratio must come out slightly **above** 2.0. The model was right and the number was early: `rchar` **was snapshotted at 974s of a 994.49s index**, 97.9% through, so 20.5s of reading never reached it. Corrected at the average rate that is +4.92 GB → **238.92 GB, ratio 2.023**, and the 2.76 GB above the floor works out at 659 KB per song for a chart plus an ini — plausible for a library of long Marathon Hero charts. The correction survives its own assumption: at *half* the average rate it still gives 236.46 GB and 2.002, above the floor. And the alternative explanation — a library that differed between probe and live run — is ruled out, because both reported `songs=4085 distinct_charts=4080`.

Same files; only the moment of reading differed. **The fix is to compute each file's SHA-256 once and hand the values to `packageHash` instead of re-reading.** It is the only one of these three with *no* semantic risk: `package_hash` is the sorted `<name>:<sha256>` construction, so identical per-file digests produce a byte-identical hash. **But it moves `package_hash`'s inputs without moving its output, and that is not allowed to ship on "the arithmetic says identical":** the hash is the `ETag` and the sync identity, and this repo's hardest rule is that it may not move. The safety story is a fixture test asserting a known folder still hashes to the same string, **red-proofed by breaking it deliberately first** — per *tune a regression test against the defect, or it is decoration*. Expected win up to ~2× for free, before any cache exists.

- *Hash files concurrently.* Mean parallelism during the 4,085-song index was **0.773** — below one — so the index is effectively single-threaded and spends about 23% of its wall time waiting. This is **independent of hashing once**: one removes half the work, the other stops the remaining half running on a single core. Bounded workers, and the array may become the ceiling, so the win is unknown until measured — but the two compose, and the ~2× above is therefore not the ceiling.
- *Persist the index and skip re-hashing unchanged songs,* keyed on path+size+mtime. This is the real answer to "scales with library size" and it is the dangerous one: `package_hash` is both the `ETag` and the sync identity, so one stale cache entry serves wrong bytes under a hash a client trusts. It needs its own ADR, its own tests and an oracle run. It must not ride along with an availability fix. The composition measured above says it would get the 481 single-file containers nearly free and have to earn the other 2,586.

What this ADR does not establish

- ~~**Where the 10m40s actually goes.**~~ **Largely answered, and the answer is in Consequences above:** full-file SHA-256 dominates and every file is hashed twice. That came from `/proc/<pid>/io` on the probe plus three `assetFor` call sites in `scan.go` — no profiler, no instrumentation. What it does **not** establish is that the index is *compute-bound*: at CPU/wall 0.773 it is single-threaded with real waiting, and an earlier draft of this section claimed otherwise. Two gaps that were open earlier are now closed by measurement. The **byte total** was summed from the catalogue at 118.08 GB, and the ratio corrected to 2.023. And **the two passes cost the same**, so eliminating one really does remove half the work:

> Six real song folders, 155,694,045 bytes, hashed and then re-hashed back to back. Per song `p1/p2` ran 0.90–1.03; overall **0.970** at ~179 MB/s then ~174 MB/s. The warm pass was *not* cheaper than the first.

That is the useful shape of the result: it does **not** settle whether the limit is SHA-256 or the array — ~175 MB/s is consistent with either, and caches were deliberately not dropped on a serving box, so pass 1 cannot be called cold. It does not need to. Whichever resource is saturated, the second pass costs what the first costs, so removing it halves the work. The page-cache worry — that pass 2 is nearly free and the prize is therefore small — is answered: it is not nearly free.

What is still *not* measured: the split between the hashing passes and the directory walk itself, and a throughput gap between the probe and the bench — **240 MB/s against 175 MB/s wall, a factor of 1.37.**

> *An earlier draft put that gap at 311 against 175, or 1.78×, and that was a unit conflation rather than a measurement.* 311 MB/s is `rchar` divided by **CPU** time — bytes per *busy* second. 175 MB/s is bytes per *wall* second. Comparing them pits a rate-while-working against a rate-including-waiting. The probe's wall figure is 238.92 GB / 994.49s = **240 MB/s**, and that is the number the bench is comparable to.

The residual 1.37× is left open with a stated likely cause rather than closed: the bench is 6 folders and 0.16 GB against the probe's 4,085 folders and 239 GB, so the file-size mix differs by orders of magnitude and large sequential stems get readahead a small sample mostly does not. That is reasoning, not a measurement, and it does not bear on the finding — which rests on the two passes costing the same, not on either absolute rate. - **That 503 is handled well by every client, as opposed to safely.** `yarg-sync` is established above to fail closed. YARG itself, reading a mirror folder on disk under increment 1, never talks to this server at all. Any third-party caller is unknown. - **The per-song cost model.** Three points, two of them with different library composition, do not separate "superlinear in count" from "the mix shifted toward loose folders". The consequence is the same either way — linear extrapolation is a floor — but the cause is not settled.
